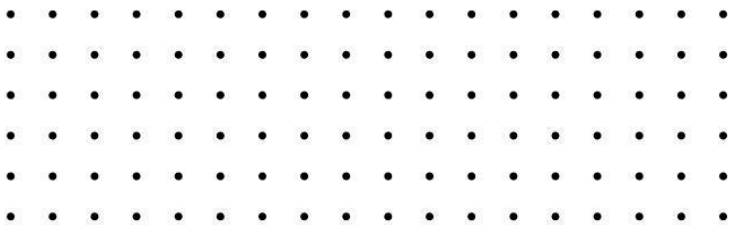


31.05.2026

SMART TECHNOHOUSE

Memoria-Manual del Proyecto

Jaime Guillermo Carrillo Salido
Programación Orientada a Objetos | UTAMED



1. Decisiones de diseño y principios SOLID

Antes de escribir una sola línea de código pensé bastante en cómo estructurar las clases para que el sistema fuera fácil de ampliar sin tener que tocar lo que ya funcionaba. La clave fue separar lo que es común a todos los dispositivos de lo que es específico de cada uno.

El punto de partida es la interfaz `IDispositivo`, que define el contrato mínimo que cualquier dispositivo tiene que cumplir: identificarse, dar un nombre legible y reportar su estado. Gracias a esto el sistema puede tratar sensores y actuadores de forma homogénea sin importar el tipo concreto que haya detrás.

De ahí cuelgan dos clases abstractas: `Sensor` y `Actuador`. `Sensor` centraliza el estado común a todos los sensores (id, nombre, unidad y último valor) y declara `actualizarValor()` como abstracto, porque cada sensor sabe cómo leer su propio dato. `Actuador` hace lo mismo para los actuadores y añade la lógica de validación de acciones y el registro en el log.

Los principios SOLID que apliqué de forma más clara fueron:

- **S (Responsabilidad única):** cada clase tiene un único motivo de cambio. El modelo guarda el estado, la vista pinta, el controlador coordina, y `GestorPersistencia` y `GestorLog` se ocupan exclusivamente de la E/S.
- **O (Abierto/Cerrado):** añadir `SensorHumedad` o `ActuadorPersiana` no obligó a tocar `Sensor` ni `Actuador`. Solo creé una clase nueva que extiende la base. Lo mismo con las reglas.
- **L (Sustitución de Liskov):** cualquier `SensorTemperatura` puede usarse donde se espera un `Sensor`. El modelo recorre `List<Sensor>` sin saber qué subclase hay.
- **D (Inversión de dependencias):** el motor de reglas depende de la interfaz `Regla`, no de ninguna regla concreta.

2. Patrones de diseño

Patrón Strategy — motor de reglas

El problema que resuelve `Strategy` aquí es claro: si metiera la lógica de todas las reglas dentro de `SmartTecnoHouse`, cada vez que quisiera añadir o cambiar una regla tendría que modificar esa clase central. La solución fue crear la interfaz `Regla` con un único método `aplicar()`. Cada regla de negocio es una clase independiente que implementa esa interfaz. `SmartTecnoHouse` mantiene una lista de reglas activas y cuando hay que ejecutarlas simplemente hace un bucle y llama a `aplicar()` en cada una, sin saber qué hace por dentro.

Patrón Modelo-Vista-Controlador (MVC)

La separación MVC fue la decisión arquitectónica más importante del proyecto. El objetivo era que el modelo no supiera nada de la interfaz gráfica y que la vista no contuviera lógica de negocio.

- **Modelo (paquete modelo):** todas las clases de dominio. No importan `javax.swing` en ningún punto.
- **Vista (VentanaPrincipal):** construye los componentes `Swing` y los refresca cuando el controlador lo pide. No decide nada por sí sola.
- **Controlador (ControladorPrincipal):** es el único que conoce a la vez al modelo y a la vista. Recibe los eventos, opera sobre el modelo y le dice a la vista que se actualice.

Patrón Singleton

SmartTecnoHouse y GestorLog son Singleton. En el caso del modelo, necesitaba garantizar que tanto el controlador como la vista accedieran siempre al mismo estado de la vivienda. En el caso del log, quería un único punto de escritura en el fichero para evitar conflictos.

3. Manual de usuario

Al arrancar la aplicación con `java -cp build App` se abre la ventana principal dividida en tres columnas y un área de log en la parte inferior.

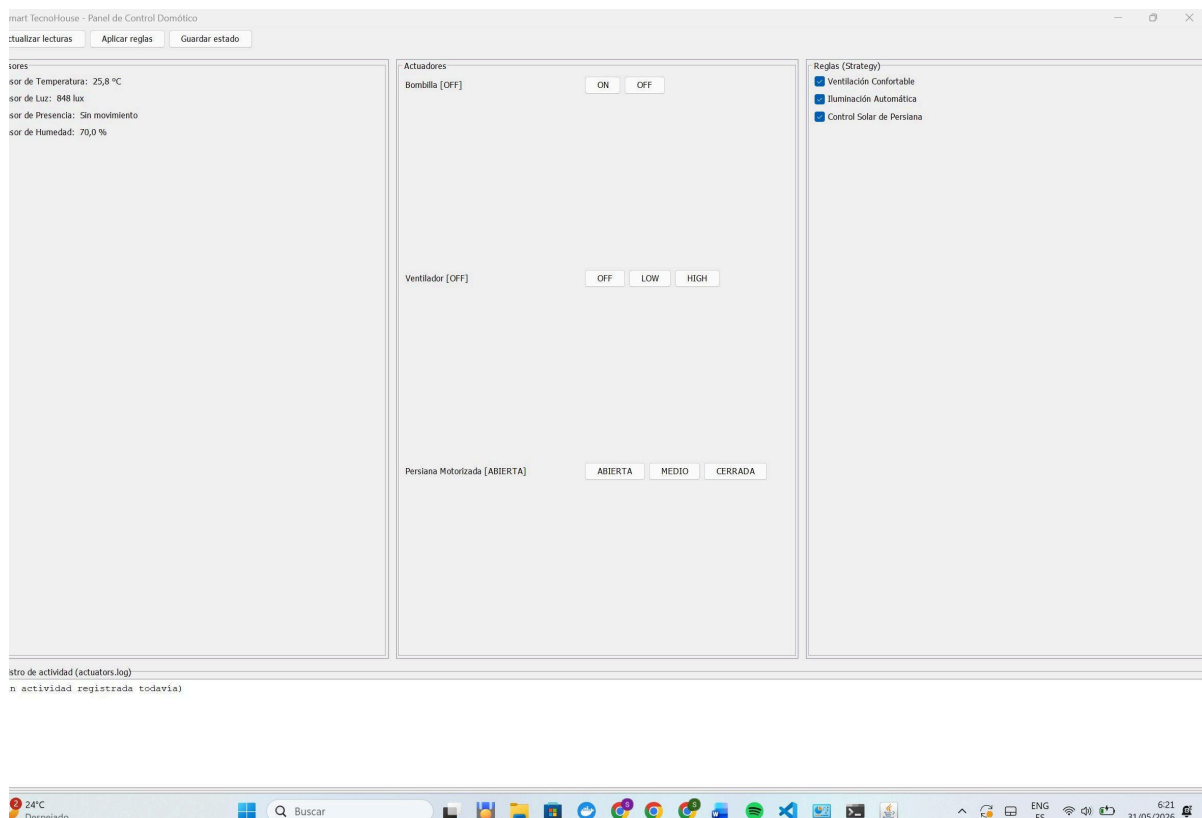


Figura 1 — Ventana principal con los cuatro sensores, tres actuadores y las tres reglas activas

Los controles disponibles son:

- Actualizar lecturas: genera nuevas lecturas simuladas para los cuatro sensores.
- Aplicar reglas: ejecuta las reglas activas sobre las lecturas actuales. Los actuadores cambian de estado automáticamente.
- Guardar estado: escribe el estado actual en `data/estado.json`. Al cerrar la ventana también se guarda automáticamente.
- Botones de actuadores: permiten cambiar el estado manualmente. Cada acción queda registrada en el log de abajo.
- Checkboxes de reglas: activan o desactivan cada regla de forma individual.



Figura 2 — Confirmación de guardado del estado en data/estado.json

```
VentanaPrincipal.java U  ControladorPrincipal.java U  Ap
data > {} estado.json > ...
1  {
2    "actuadores": {
3      "bulb": "OFF",
4      "fan": "OFF",
5      "persiana": "ABIERTA"
6    },
7    "reglas": {
8      "Ventilación Confortable": true,
9      "Iluminación Automática": true,
10     "Control Solar de Persiana": true
11   }
12 }
13 |
```

Figura 3 — Contenido del fichero data/estado.json con el estado de actuadores y reglas

4. Ampliaciones: nuevo sensor y nuevo actuador

Sensor de Humedad (SensorHumedad)

El sensor de humedad mide la humedad relativa del aire en porcentaje, con valores simulados entre el 25% y el 80%. Para añadirlo solo tuve que crear la clase SensorHumedad extendiendo Sensor e implementar actualizarValor(). No hizo falta modificar ninguna clase existente, lo que demuestra en la práctica el principio Abierto/Cerrado.

Persiana Motorizada (ActuadorPersiana)

La persiana motorizada tiene tres posiciones: ABIERTA, MEDIO y CERRADA. Añadirla consistió en crear `ActuadorPersiana` extendiendo `Actuador` y definir `getAccionesPosibles()`. Nada más en el sistema tuvo que cambiar.

Para demostrar que estos dos dispositivos nuevos realmente encajan en el sistema, creé la regla `ReglaPersianaSolar`, que usa el sensor de luz para regular la persiana automáticamente: con más de 800 lux la cierra del todo, entre 400 y 800 la deja a medias y con menos de 400 la abre para aprovechar la luz natural.